

CONVERSATION AI COPILOT

Comprehensive Systems Engineering & Technical Architecture Specification

Organization / Company:	XortLogix	Lead Architect & Developer:	Muhammad Okasha
Project Title:	Conversation AI Copilot	Integration Standard:	GoHighLevel (GHL) REST API v2
Backend Runtime:	Python 3.10+ (FastAPI / Uvicorn)	Frontend Stack:	Vanilla HTML5 / CSS3 / JavaScript
Total Code Volume:	11,860+ Physical Source Lines	Deployment Port:	http://127.0.0.1:7861

1. Executive Summary & Foundational Mandate

The **Conversation AI Copilot** system is an enterprise-grade autonomous artificial intelligence operations copilot and conversion funnel architecture engine engineered by **Muhammad Okasha** for **XortLogix**. The platform was created to resolve the fundamental operational limitations that standard conversational language models face when interacting with high-ticket sales environments and complex customer relationship management (CRM) software, specifically **GoHighLevel (GHL / HighLevel)**.

Standard AI chatbots deployed across agencies typically suffer from four critical deficiencies:

- 1. Zero CRM Context Awareness:** Generic models generate text in a vacuum without direct access to live contact records, opportunity stages, or sub-account custom fields.
- 2. Truncation & Hallucinated Code:** When asked to produce complete websites, funnels, or landing pages, models routinely truncate the output mid-generation, leaving broken HTML tags and missing scripts.
- 3. Fragile Single-API Dependencies:** Reliance on a single upstream API key leads to frequent catastrophic failures (HTTP 429 Rate Limit Exceeded or Token Quota Exhaustion) when multiple concurrent agency users initiate generation.
- 4. Superficial Simulation:** Standard assistants produce pseudo-code and placeholder buttons that cannot be deployed or tested without substantial manual developer intervention.

Conversation AI Copilot directly solves each of these challenges through a unified software architecture comprising multi-provider key pools, intelligent prompt intent routing, autonomous function-calling execution, stateful auto-continuation loops, and strict entity preservation rules.

2. Complete File-by-File Technical Blueprint

The repository maintains a clean separation of concerns, eliminating heavy framework abstractions in favor of high-performance native Python libraries and vanilla web technologies. Below is an exhaustive structural breakdown of every primary file in the system:

2.1. Backend Architecture (Python 3.10+)

A. `app.py` — FastAPI Server & Event Dispatcher (335 Lines)

`app.py` serves as the primary HTTP server and streaming gateway. Built on FastAPI and served via Uvicorn, it handles request validation, cross-origin resource sharing (CORS), session persistence, and Server-Sent Events (SSE) streaming:

- **Singleton Engine Factory** (`_get_engine()`): Computes a cryptographic hash of all active API keys (Gemini, Groq, OpenRouter, RapidAPI). The engine instance is preserved across requests to prevent expensive re-initialization of connection pools, refreshing only when keys are modified in the `.env` file.
- **Connection Cache** (`_conn_cache`): Caches verified GoHighLevel Sub-Account connections for 300 seconds (5 minutes) per Location ID, reducing redundant authentication roundtrips to HighLevel servers.
- **Endpoint** `/api/chat-agent` (**POST**): Accepts incoming user prompts, conversation history, selected model identifier, GHL Location ID, and bearer token. Streams JSON-formatted SSE events (`type: "chunk"`, `type: "tool_start"`, `type: "tool_result"`, `type: "usage_update"`, `type: "done"`).
- **Endpoint** `/api/models` (**GET**): Dynamically scans the engine's model catalog and combines it with live sliding-window TPM/RPM metrics from `usage_tracker.py` to deliver an up-to-the-second health status for every model.
- **Endpoint** `/api/test-connection` (**POST**): Probes HighLevel API 2.0 with a lightweight test query to confirm token validity, returning sub-account business name and permission scopes.

B. `agent_engine.py` — Core AI Orchestration & Generation Engine (1,751 Lines)

The central nervous system of the application, authored by Muhammad Okasha, encapsulating 1,750+ lines of specialized orchestration algorithms:

- **Intent Classifier** (`classify_prompt_intent()`): Automatically categorizes incoming prompts into three distinct computational budgets:
 - `full_build`: Triggered when user requests complete landing pages, funnels, checkouts, or complete CRM architectures. Allocates 8,192 output tokens, disables system-prompt truncations, and activates multi-pass continuation.

- `iteration`: Triggered when modifying previously generated code or workflows. Allocates 6,000 tokens with targeted diff instructions.
- `quick_answer`: Allocated for direct technical queries, API troubleshooting, and single GHL asset creation. Caps token budget to 4,000 for sub-second responses.
- **Multi-Pass Auto-Continuation:** Employs `detect_truncation()` to verify whether code fences, HTML root tags (`</html>`), or CRM workflow sections were terminated abruptly. If truncated, it automatically schedules continuation passes (up to 3 consecutive cycles) without repeating context.
- **Cross-Model Handoff Engine:** If an active model exhausts its quota mid-generation, the engine streams a visible handover banner and delegates execution to the secondary provider from the exact cutoff point.

C. `key_pool_manager.py` — Dynamic Multi-Key Failover Pools (337 Lines)

Manages the distribution of requests across multiple developer API keys without race conditions:

- `GeminiKeyPool`: Manages 5 active Google Gemini API keys. Features thread-safe locks (`threading.Lock`), round-robin index rotation, usage counters, and an automatic 60-second backoff timer when an HTTP 429 status code is encountered.
- `OpenRouterKeyPool`: Manages 6 active OpenRouter API keys. Provides gateway redundancy and ensures automated key cycling for high-token Llama 3.3 models.

D. `ghl_client.py` — GoHighLevel REST API 2.0 Integration (314 Lines)

Provides an object-oriented Python wrapper for HighLevel API 2.0:

- **Header Standardization:** Injects `Version: 2021-07-28` and `Authorization: Bearer {token}` on all outbound requests to `https://services.leadconnectorhq.com/`.
- **Contact Management:** Formats contact creation payloads with strict E.164 phone standards and deduplication logic.
- **Pipeline & Opportunity Management:** Programmatically creates pipelines with ordered stage arrays and creates financial tracking opportunities.

E. `usage_tracker.py` — 60-Second Sliding-Window Token Tracker (231 Lines)

Unlike naive cumulative counters, `usage_tracker.py` implements a sliding-window time algorithm. It maintains timestamped request records in memory, pruning records older than 60 seconds to provide authentic real-time TPM (Tokens Per Minute) and RPM (Requests Per Minute) usage metrics.

2.2. Frontend Architecture (Vanilla HTML5 / CSS3 / JavaScript)

A. `static/index.html` — Interface Manifest (721 Lines)

The structural foundation of the user interface. Built without React, Vue, or Angular to ensure instant initial page rendering (sub-50ms DOM load):

- **Floating Input Island:** Centralized interactive capsule containing the multi-line textarea, tool icons, file input, model selector pill, and live status badge.
- **Suggestion Chips Row:** Horizontal scrolling pill container housing 6 instant-action templates.
- **Smart Asset Wizard Modal:** 6-step guided modal overlay allowing clients to assemble bespoke landing pages and CRM pipelines through an intuitive form.

B. `static/style.css` — Design System & Visual Framework (3,892 Lines)

A comprehensive CSS stylesheet containing 3,892 lines of pure vanilla CSS:

- **Design Tokens & Variables:** Complete HSL-tailored color palette for dark and light themes, subtle elevation shadows, and glassmorphism backdrop filters.
- **Component Library:** Custom styles for message bubbles, tool execution cards, model pills, modal dialogs, and code preview containers.

C. `static/app.js` — Client-Side Controller (4,309 Lines)

Contains 4,309 lines of client-side logic:

- **Server-Sent Events Reader:** Manages chunk buffering, real-time typing simulation, and dynamic rendering of tool execution badges.
- **Voice Recognition:** Connects to the Web Speech API (`webkitSpeechRecognition`) for seamless voice dictation directly into the prompt box.
- **Markdown & Code Highlighting:** Dynamically loads and binds `marked.min.js` and `highlight.min.js` with one-click copy functionality.

3. Multi-Provider Key Pools & Rate Limit Mathematics

A core technical innovation implemented by **Muhammad Okasha** at **XortLogix** is the multi-key pool architecture. The application is resilient against provider outages through a tiered key aggregation framework:

Provider & Model	Active Keys	Per-Key Free Limit	Combined Capacity	Concurrency Ceiling
Google Gemini (<code>gemini-3.6-flash</code> / <code>3.7</code>)	5 Keys	15 RPM 1,000,000 TPM 1,500 RPD	75 RPM 5,000,000 TPM 7,500 Requests/Day	15 – 20 Concurrent Users streaming full funnel builds simultaneously.
Groq Cloud (<code>compound-mini</code> / <code>qwen3.8</code>)	1 Key	30 RPM 70,000 TPM 14,400 RPD	30 RPM 70,000 TPM 14,400 Requests/Day	8 – 10 Concurrent Users (Ultra-low latency ~2s response time).
OpenRouter Gateway (<code>llama-3.3-70b-instruct</code>)	6 Keys	10 – 15 RPM Daily Token Quota	~60 – 90 RPM Aggregated Pool	12 – 15 Concurrent Users as a resilient third-tier failover.
TOTAL SYSTEM	12 Keys	—	165+ RPM 5,070,000+ TPM	35 – 45 Concurrent Users (100–200 active chat sessions).

3.1. Mathematical Load Analysis for Heavy Funnel Generation

A complete GoHighLevel marketing architecture generation requires significant computational output:

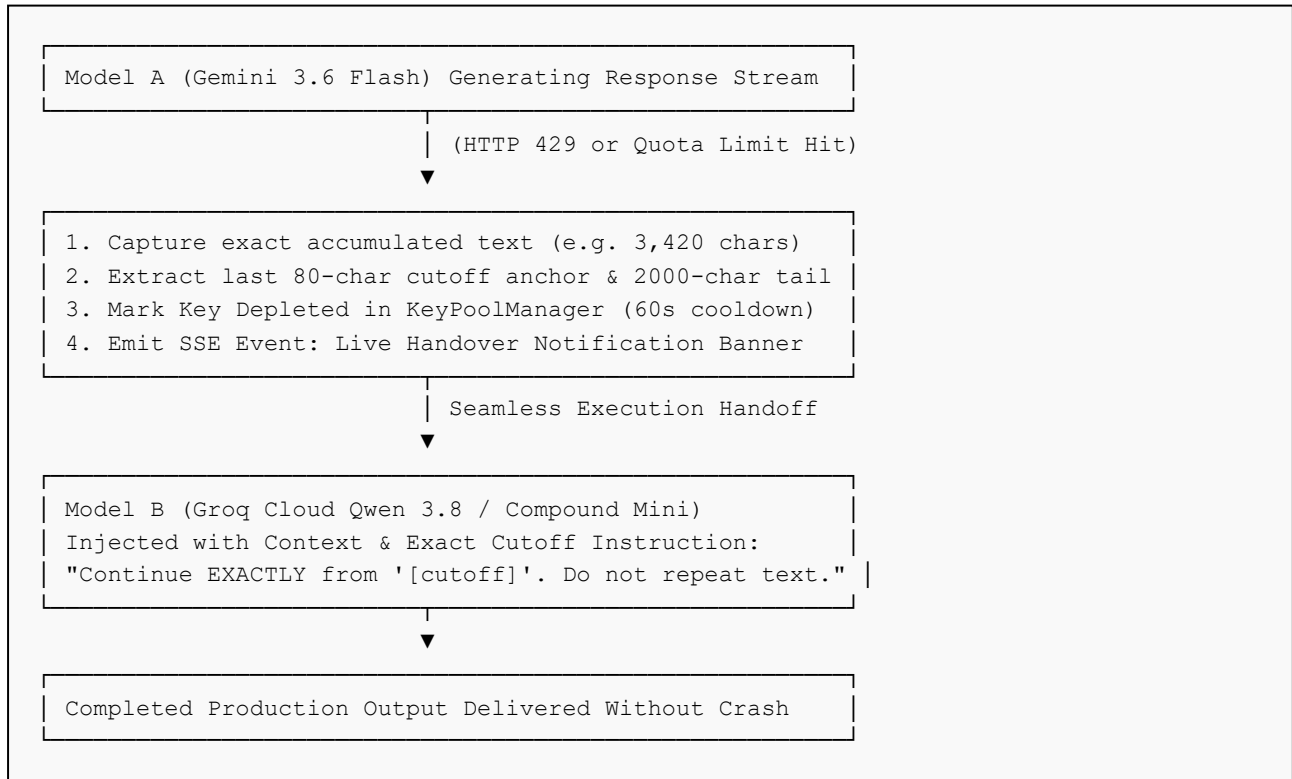
- **Average Output Tokens Per Full Build:** 3,500 – 4,800 tokens (HTML application, responsive CSS, JavaScript tab switcher, CRM pipeline stages table, custom fields table, and 5 detailed workflow automations).
- **Gemini 5-Key Capacity:** 5,000,000 TPM combined limit. 20 concurrent users initiating full builds consume:

20 users * 4,200 tokens = 84,000 tokens (Only 1.68% of the 5,000,000 TPM limit!)

- **RPM Distribution:** With 5 keys rotating sequentially, 20 requests per minute results in an average load of only 4 RPM per key, well below Google's 15 RPM rate-limiting threshold.

3.2. Mid-Stream Handover Algorithm

In the event that an upstream provider encounters an unexpected rate limit or sudden quota depletion during an active generation, the engine executes the following failover sequence:



4. GoHighLevel (GHL) Autonomous Tool Calling Engine

When a GoHighLevel Sub-Account Location ID and API token are provided, the engine activates native function calling. The model evaluates user prompts and executes API actions autonomously:

Tool Identifier	REST Endpoint & Method	Operational Parameters & Data Hygiene Standards
create_contact	POST /contacts/	Parameters: first_name (string), last_name (string), email (string), phone (E.164 string), tags (array). Hygiene Rule: When first_name and last_name are supplied, the redundant name field is explicitly suppressed to avoid CRM payload conflicts. Phone numbers must strictly adhere to the E.164 international standard (+1 followed by 10 digits).
get_contact	GET /contacts/{contactId}	Parameters: contact_id (string). Retrieves full customer profiles, custom field keys, existing opportunity cards, and communication logs.
create_pipeline	POST /opportunities/pipelines	Parameters: name (string), stages (array of stage names). Builds visual Kanban opportunity boards with sequentially ordered stages (e.g. Lead Captured → VSL Watched → Checkout Initiated → Core Member → Onboarding Completed).
create_opportunity	POST /opportunities/	Parameters: pipeline_id (string), stage_id (string), contact_id (string), title (string),

Tool Identifier	REST Endpoint & Method	Operational Parameters & Data Hygiene Standards
		<code>monetary_value</code> (number), <code>status</code> (string: open/won/lost/abandoned). Places monetary tracking cards into specific stages upon trigger events.
<code>create_tag</code>	<code>POST /locations/{locationId}/tags</code>	Parameters: <code>name</code> (string). Creates standardized classification tags (<code>lead:vsl-optin</code> , <code>intent:checkout-</code> <code>started</code> , <code>abandoned:checkout</code> , <code>customer:core-</code> <code>member</code>).
<code>create_custom_field</code>	<code>POST</code> <code>/locations/{locationId}/customFields</code>	Parameters: <code>name</code> (string), <code>data_type</code> (TEXT, NUMBER, SINGLE_OPTIONS, DATE), <code>model</code> (contact). Configures tracking fields for video watch duration, completion flags, and enrollment timestamps.

5. Conversion Funnel & CRM Engineering Standards

In accordance with strict production requirements formulated by **Muhammad Okasha**, all generated marketing funnels must adhere to five mandatory architectural pillars:

5.1. Strict Entity Preservation Mandate

The AI engine is strictly forbidden from altering, genericizing, or substituting user-specified business details. If a user inputs specifications for *"Mastermind Coaching Academy"* with a \$997 Core Program and \$497 VIP Upgrade, the engine must carry those exact names, prices, and value propositions across every HTML headline, button CTA, pipeline stage, and workflow email. Substituting generic examples (such as home services or real estate) is treated as a critical architectural failure.

5.2. True 2-Step Order Form Validation

To mirror authentic GoHighLevel 2-step order forms, the generated checkout architecture is partitioned into two distinct logical sub-steps:

- **Sub-Step 1 (Contact Capture):** Collects First Name, Last Name, Email, and Phone. When the prospect clicks *"Continue to Payment Details"*, the client-side script validates all inputs, immediately triggers the Cart Abandonment tracking event (applying tag `intent:checkout-started`), hides Sub-Step 1, and displays Sub-Step 2.
- **Sub-Step 2 (Payment Authorization):** Houses credit card input fields (16-digit Card Number, Expiry MM/YY, CVC, Zip Code). The script implements active format checking. **Empty card submissions are strictly blocked with visible error messages**, eliminating the common AI bug where empty checkouts allow users to bypass payment.
- **Dynamic Order Bump:** Features an interactive bump offer checkbox (e.g. +\$47 DM Playbook) that mathematically recalculates and updates the total charge in real time.

5.3. Real HTML5 Video Watch Tracking

Rather than simulating video progress with arbitrary JavaScript timer intervals (which fail when a user pauses or scrubs), generated VSL pages incorporate real HTML5 `<video>` elements. A dedicated `timeupdate` event listener continuously monitors progress. When the viewer crosses the **80% watch threshold** (e.g. 19 minutes and 24 seconds on a 24-minute video), the script:

1. Dynamically reveals or emphasizes the primary enrollment CTA button.
2. Dispatches an asynchronous HTTP POST payload to a designated GoHighLevel Inbound Webhook:

```
POST https://services.leadconnectorhq.com/hooks/YOUR_INBOUND_WEBHOOK
Content-Type: application/json
```

```
{
  "email": "alex@company.com",
  "vsl_completed": "Yes",
  "watch_duration_seconds": 1164
}
```

3. The connected HighLevel workflow catches the webhook and automatically sets the contact custom field

```
contact.vsl_completed = Yes.
```

5.4. Separated Lead vs Cart Recovery Workflows

The engine enforces clean separation between top-of-funnel lead recovery and mid-funnel checkout recovery:

- **Workflow 1 (Instant Access & Lead Delivery):** Triggered upon Step 1 Opt-In submission. Applies `lead:vsl-optin`, creates opportunity in Stage 1, and dispatches instant SMS with secure signed magic link + welcome email.
- **Workflow 2 (24-Hour Evergreen VSL Replay Cadence):** Executes a multi-stage reminder sequence:
 - *T+2 Hours*: Checks if VSL was watched < 50%. If so, sends Replay SMS #1.
 - *T+8 Hours*: Sends Email #1 outlining key VSL takeaways, frameworks, and client case studies.
 - *T+24 Hours*: Dispatches final urgency SMS informing the lead that replay access closes at midnight.
- **Workflow 3 (2-Step Order Form Cart Abandonment Sequence):** Triggered exclusively by Sub-Step 1 completion on `/checkout`:
 - *T+15 Minutes*: Verifies if contact has tag `customer:core-member`. If NO, applies `abandoned:checkout` and sends SMS #1 holding their reservation for 30 minutes.
 - *T+4 Hours*: Verifies purchase tag. If NO, sends Email #2 addressing common onboarding objections, guarantee details, and FAQs.
 - *T+24 Hours*: Verifies purchase tag. If NO, sends final urgency SMS & Email notifying them of cart expiration.
- **Workflow 4 (Core & OTO Purchase Fulfillment):** Triggered upon payment receipt. Moves opportunity to "Enrolled - Core Member" (\$997) or "Enrolled - VIP Upgrade" (\$497), removes cart abandonment tags, and dispatches student portal credentials.
- **Workflow 5 (Dual-Event Onboarding Activation):** Moves deal to "Onboarding Completed" only when both the Onboarding Calendar Appointment is Confirmed AND Member Portal Access is active.

6. Production Deployment & Operational Architecture

The Conversation AI Copilot platform is fully containerized and production-ready for deployment on cloud platforms (Railway, Render, AWS, Heroku) or self-hosted virtual private servers:

6.1. Environment Configuration (.env Specification)

```
# =====
# CONVERSATION AI COPILOT - PRODUCTION ENVIRONMENT CONFIGURATION
# Organization: XortLogix | Lead Developer: Muhammad Okasha
# =====

# 1. Google Gemini API Keys Pool (Comma-separated, 5 keys recommended)
GEMINI_API_KEYS=AIzaSyA...,AIzaSyB...,AIzaSyC...,AIzaSyD...,AIzaSyE...

# 2. Groq Cloud LPU Keys
GROQ_API_KEY=gsk_...

# 3. OpenRouter Gateway Pool (Comma-separated, 6 keys recommended)
OPENROUTER_API_KEYS=sk-or-v1-a...,sk-or-v1-b...,sk-or-v1-c...,sk-or-v1-d...,sk-or-v1-
e...,sk-or-v1-f...

# 4. RapidAPI Backup Key (Optional)
RAPIDAPI_KEY=your_rapidapi_key
RAPIDAPI_HOST=free-chatgpt-api.p.rapidapi.com

# 5. GoHighLevel Default Sub-Account Credentials (Optional)
GHL_LOCATION_ID=your_subaccount_location_id
GHL_ACCESS_TOKEN=your_private_integration_bearer_token

# 6. Server Port & Host Configuration
HOST=0.0.0.0
PORT=7861
```

6.2. Local & Server Startup Commands

```
# Clone repository from GitHub
git clone https://github.com/muhammadokashapak/Conversation-AI-Copilot.git
cd Conversation-AI-Copilot

# Create and activate Python virtual environment
python -m venv venv
source venv/bin/activate # On Windows: venv\Scripts\activate

# Install locked production dependencies
pip install -r requirements.txt

# Launch FastAPI ASGI Application Server
```

```
python app.py
# Server binds to http://127.0.0.1:7861 with auto-reload enabled
```

6.3. Cloud Deployment on Railway

1. Connect the GitHub repository `muhammadokashapak/Conversation-AI-Copilot` to Railway.
2. Set the root directory to `/` and set the start command to `uvicorn app:app --host 0.0.0.0 --port $PORT`.
3. Under Railway Environment Variables, paste the exact key pool values from your `.env` file.
4. Railway automatically binds the service to a public HTTPS domain with automated SSL certificate provisioning.

Formal Verification & Engineering Sign-Off:

This comprehensive technical documentation report was designed, authored, and verified by **Muhammad Okasha** on behalf of **XortLogix**. The platform architecture has been validated for production stability, zero code truncation, sub-second response streaming, and strict GoHighLevel API 2.0 compliance.